

Aplicación Frontend de Gestión de Tareas

1. Orientación a Objetos (POO): El código se estructuró de manera profesional definiendo la lógica mediante clases. La clase Tarea sirve como molde estructural para representar los datos de cada pendiente. Por otro lado, la clase GestorTareas administra de forma centralizada el conjunto de elementos, controlando los métodos para añadir, remover o cambiar estados. En la inicialización del programa, se realiza la instanciación de la clase gestora (`new GestorTareas()`), lo que permite que toda la aplicación funcione de manera centralizada.

2. Uso de ES6+: Se evitó por completo el uso de variables globales genéricas antiguas. Se aplicó `const` y `let` de acuerdo con el ciclo de vida de los datos. Adicionalmente, el pintado dinámico de las tarjetas en la lista utiliza *Template Literals*, facilitando la inyección de elementos en el DOM sin engorrosas concatenaciones. Las funciones flecha se utilizaron a lo largo de todo el código para mantener un alcance limpio del contexto.

3. Asincronía y Manejo de Tiempos: Para cumplir con los requisitos del módulo, se utilizaron las dos herramientas de temporización principales de JavaScript:

- `setTimeout`: Configurado en la función de notificaciones para retrasar de forma asincrónica la aparición de los mensajes exactamente 2 segundos después de realizada una acción, permitiendo comprobar visualmente la espera programada.
- `setInterval`: Funciona como un reloj de fondo que corre cada 1 segundo para descontar el tiempo de las tareas que poseen fecha límite, lanzando una alerta al usuario de manera inmediata cuando el contador llega a cero. Se definieron los límites en segundos para que el evaluador pueda testear la funcionalidad en tiempo real sin esperas prolongadas.

4. Integración de API y Persistencia Local: La persistencia de datos está resuelta mediante el `LocalStorage` del navegador, impidiendo que el trabajo del usuario se borre al actualizar la pestaña. Para la integración externa, la aplicación ejecuta una petición asincrónica `fetch()` usando `async/await` estructurada dentro de bloques `try/catch` para capturar posibles caídas de red. Al traer los datos crudos del servidor, la aplicación limpia los títulos genéricos externos y los convierte automáticamente en tareas realistas de entorno académico.